

18.1-Gradient Boosting Classification

September 23, 2025

1 Gradient Boosting Classification Implementation

```
[33]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import plotly.express as px
import warnings
warnings.filterwarnings('ignore')
import seaborn as sns
```

```
[34]: df = pd.read_csv('../0-Dataset/Travel.csv')
df.head()
```

```
[34]:
```

	CustomerID	ProdTaken	Age	TypeofContact	CityTier	DurationOfPitch	\
0	200000	1	41.0	Self Enquiry	3	6.0	
1	200001	0	49.0	Company Invited	1	14.0	
2	200002	1	37.0	Self Enquiry	1	8.0	
3	200003	0	33.0	Company Invited	1	9.0	
4	200004	0	NaN	Self Enquiry	1	8.0	

	Occupation	Gender	NumberOfPersonVisiting	NumberOfFollowups	\
0	Salaried	Female	3	3.0	
1	Salaried	Male	3	4.0	
2	Free Lancer	Male	3	4.0	
3	Salaried	Female	2	3.0	
4	Small Business	Male	2	3.0	

	ProductPitched	PreferredPropertyStar	MaritalStatus	NumberOfTrips	\
0	Deluxe	3.0	Single	1.0	
1	Deluxe	4.0	Divorced	2.0	
2	Basic	3.0	Single	7.0	
3	Basic	3.0	Divorced	2.0	
4	Basic	4.0	Divorced	1.0	

	Passport	PitchSatisfactionScore	OwnCar	NumberOfChildrenVisiting	\
0	1	2	1	0.0	

1	0	3	1	2.0
2	1	3	0	0.0
3	1	5	1	1.0
4	0	5	1	0.0

	Designation	MonthlyIncome
0	Manager	20993.0
1	Manager	20130.0
2	Executive	17090.0
3	Executive	17909.0
4	Executive	18468.0

1.1 Data Cleaning

```
[35]: df.isnull().sum()
```

```
[35]: CustomerID          0
      ProdTaken          0
      Age              226
      TypeofContact      25
      CityTier           0
      DurationOfPitch    251
      Occupation         0
      Gender             0
      NumberOfPersonVisiting  0
      NumberOfFollowups   45
      ProductPitched      0
      PreferredPropertyStar  26
      MaritalStatus       0
      NumberOfTrips      140
      Passport           0
      PitchSatisfactionScore  0
      OwnCar              0
      NumberOfChildrenVisiting  66
      Designation         0
      MonthlyIncome      233
      dtype: int64
```

```
[36]: df['Gender'].value_counts()
```

```
[36]: Gender
      Male      2916
      Female    1817
      Fe Male     155
      Name: count, dtype: int64
```

```
[37]: df['Gender'] = df['Gender'].replace('Fe Male', 'Female')
```

```
[38]: df['Gender'].value_counts()
```

```
[38]: Gender
Male      2916
Female    1972
Name: count, dtype: int64
```

```
[39]: df['MaritalStatus'].value_counts()
```

```
[39]: MaritalStatus
Married      2340
Divorced      950
Single        916
Unmarried     682
Name: count, dtype: int64
```

```
[40]: df['MaritalStatus'] = df['MaritalStatus'].replace('Single', 'Unmarried')
```

```
[41]: df['MaritalStatus'].value_counts()
```

```
[41]: MaritalStatus
Married      2340
Unmarried    1598
Divorced      950
Name: count, dtype: int64
```

```
[42]: df['TypeofContact'].value_counts()
```

```
[42]: TypeofContact
Self Enquiry      3444
Company Invited    1419
Name: count, dtype: int64
```

```
[43]: df.head()
```

```
[43]:
```

	CustomerID	ProdTaken	Age	TypeofContact	CityTier	DurationOfPitch	\
0	200000	1	41.0	Self Enquiry	3	6.0	
1	200001	0	49.0	Company Invited	1	14.0	
2	200002	1	37.0	Self Enquiry	1	8.0	
3	200003	0	33.0	Company Invited	1	9.0	
4	200004	0	NaN	Self Enquiry	1	8.0	

	Occupation	Gender	NumberOfPersonVisiting	NumberOfFollowups	\
0	Salaried	Female	3	3.0	
1	Salaried	Male	3	4.0	
2	Free Lancer	Male	3	4.0	
3	Salaried	Female	2	3.0	
4	Small Business	Male	2	3.0	

	ProductPitched	PreferredPropertyStar	MaritalStatus	NumberOfTrips	\
0	Deluxe	3.0	Unmarried	1.0	
1	Deluxe	4.0	Divorced	2.0	
2	Basic	3.0	Unmarried	7.0	
3	Basic	3.0	Divorced	2.0	
4	Basic	4.0	Divorced	1.0	

	Passport	PitchSatisfactionScore	OwnCar	NumberOfChildrenVisiting	\
0	1	2	1	0.0	
1	0	3	1	2.0	
2	1	3	0	0.0	
3	1	5	1	1.0	
4	0	5	1	0.0	

	Designation	MonthlyIncome
0	Manager	20993.0
1	Manager	20130.0
2	Executive	17090.0
3	Executive	17909.0
4	Executive	18468.0

```
[44]: ## Handling the nan values
features_with_na = [features for features in df.columns if df[features].
    isnull().sum()>=1]
for feature in features_with_na:
    print(feature, np.round(df[feature].isnull().mean()*100, 5), '% missing_
    values')
```

```
Age 4.62357 % missing values
TypeofContact 0.51146 % missing values
DurationOfPitch 5.13502 % missing values
NumberOfFollowups 0.92062 % missing values
PreferredPropertyStar 0.53191 % missing values
NumberOfTrips 2.86416 % missing values
NumberOfChildrenVisiting 1.35025 % missing values
MonthlyIncome 4.76678 % missing values
```

```
[45]: df[features_with_na].select_dtypes(exclude='object').describe()
```

```
[45]:
```

	Age	DurationOfPitch	NumberOfFollowups	PreferredPropertyStar	\
count	4662.000000	4637.000000	4843.000000	4862.000000	
mean	37.622265	15.490835	3.708445	3.581037	
std	9.316387	8.519643	1.002509	0.798009	
min	18.000000	5.000000	1.000000	3.000000	
25%	31.000000	9.000000	3.000000	3.000000	
50%	36.000000	13.000000	4.000000	3.000000	
75%	44.000000	20.000000	4.000000	4.000000	

max	61.000000	127.000000	6.000000	5.000000
-----	-----------	------------	----------	----------

	NumberOfTrips	NumberOfChildrenVisiting	MonthlyIncome
count	4748.000000	4822.000000	4655.000000
mean	3.236521	1.187267	23619.853491
std	1.849019	0.857861	5380.698361
min	1.000000	0.000000	1000.000000
25%	2.000000	1.000000	20346.000000
50%	3.000000	1.000000	22347.000000
75%	4.000000	2.000000	25571.000000
max	22.000000	3.000000	98678.000000

```
[46]: df.Age.fillna(df.Age.median(), inplace=True)
df.TypeOfContact.fillna(df.TypeOfContact.mode()[0], inplace=True)
df.DurationOfPitch.fillna(df.DurationOfPitch.median(), inplace=True)
df.NumberOfFollowups.fillna(df.NumberOfFollowups.mode()[0], inplace=True)
df.PreferredPropertyStar.fillna(df.PreferredPropertyStar.mode()[0],
    ↪inplace=True)
df.NumberOfTrips.fillna(df.NumberOfTrips.median(), inplace=True)
df.NumberOfChildrenVisiting.fillna(df.NumberOfChildrenVisiting.mode()[0],
    ↪inplace=True)
df.MonthlyIncome.fillna(df.MonthlyIncome.median(), inplace=True)
```

```
[47]: df.isnull().sum()
```

```
[47]: CustomerID          0
ProdTaken                0
Age                     0
TypeofContact           0
CityTier                0
DurationOfPitch         0
Occupation              0
Gender                  0
NumberOfPersonVisiting  0
NumberOfFollowups       0
ProductPitched          0
PreferredPropertyStar    0
MaritalStatus           0
NumberOfTrips           0
Passport                0
PitchSatisfactionScore   0
OwnCar                  0
NumberOfChildrenVisiting 0
Designation             0
MonthlyIncome           0
dtype: int64
```

```
[48]: df.drop('CustomerID', axis=1, inplace=True)
```

1.2 Feature Engineering

```
[49]: df.head()
```

```
[49]:
```

	ProdTaken	Age	TypeofContact	CityTier	DurationOfPitch	\
0	1	41.0	Self Enquiry	3	6.0	
1	0	49.0	Company Invited	1	14.0	
2	1	37.0	Self Enquiry	1	8.0	
3	0	33.0	Company Invited	1	9.0	
4	0	36.0	Self Enquiry	1	8.0	

	Occupation	Gender	NumberOfPersonVisiting	NumberOfFollowups	\
0	Salaried	Female	3	3.0	
1	Salaried	Male	3	4.0	
2	Free Lancer	Male	3	4.0	
3	Salaried	Female	2	3.0	
4	Small Business	Male	2	3.0	

	ProductPitched	PreferredPropertyStar	MaritalStatus	NumberOfTrips	\
0	Deluxe	3.0	Unmarried	1.0	
1	Deluxe	4.0	Divorced	2.0	
2	Basic	3.0	Unmarried	7.0	
3	Basic	3.0	Divorced	2.0	
4	Basic	4.0	Divorced	1.0	

	Passport	PitchSatisfactionScore	OwnCar	NumberOfChildrenVisiting	\
0	1	2	1	0.0	
1	0	3	1	2.0	
2	1	3	0	0.0	
3	1	5	1	1.0	
4	0	5	1	0.0	

	Designation	MonthlyIncome
0	Manager	20993.0
1	Manager	20130.0
2	Executive	17090.0
3	Executive	17909.0
4	Executive	18468.0

```
[50]: df['TotalVisiting'] =
    ↪ df['NumberOfChildrenVisiting'] + df['NumberOfPersonVisiting']
df.drop(['NumberOfChildrenVisiting', 'NumberOfPersonVisiting'], axis=1,
    ↪ inplace=True)
```

```
[51]: # get all the numeric feature
num_features= [feature for feature in df.columns if df[feature].dtype !='0']
print('Number of Numerical Features: ', len(num_features))
```

Number of Numerical Features: 12

```
[52]: # get all the categorical feature
num_features= [feature for feature in df.columns if df[feature].dtype =='0']
print('Number of Categorical Features: ', len(num_features))
```

Number of Categorical Features: 6

```
[53]: # get all the discrete feature
discrete_features= [feature for feature in num_features if len(df[feature].
    ↳unique()) <=25]
print('Number of Discrete Features: ', len(discrete_features))
```

Number of Discrete Features: 6

```
[54]: # Continuous Features
continuous_features = [feature for feature in num_features if feature not in
    ↳discrete_features]
print('number of Continuous Features: ', len(continuous_features))
```

number of Continuous Features: 0

2 Model Building

```
[55]: from sklearn.model_selection import train_test_split
X = df.drop(['ProdTaken'], axis=1)
y = df['ProdTaken']
```

```
[56]: y.value_counts()
```

```
[56]: ProdTaken
0    3968
1     920
Name: count, dtype: int64
```

```
[57]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↳random_state=42)
X_train.shape, X_test.shape
```

```
[57]: ((3910, 17), (978, 17))
```

```
[58]: # Create Column Transformer with 3 types of transformers

cat_features = X.select_dtypes(include='object').columns
num_features = X.select_dtypes(exclude='object').columns
```

```

# we use the column Transformer for transform the columns with combining
↳ multiple techniques

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

numeric_transformer = StandardScaler()
oh_transformer = OneHotEncoder(drop = 'first') # only (n-1) columns are required

preprocessor = ColumnTransformer(
    [
        ("OneHotEncoder", oh_transformer, cat_features),
        ('StandardScaler', numeric_transformer, num_features)
    ]
)

```

```
[59]: print(preprocessor)
```

```

ColumnTransformer(transformers=[('OneHotEncoder', OneHotEncoder(drop='first'),
                                Index(['TypeofContact', 'Occupation', 'Gender',
'ProductPitched',
    'MaritalStatus', 'Designation'],
    dtype='object')),
                                ('StandardScaler', StandardScaler(),
                                Index(['Age', 'CityTier', 'DurationOfPitch',
'NumberOfFollowups',
    'PreferredPropertyStar', 'NumberOfTrips', 'Passport',
    'PitchSatisfactionScore', 'OwnCar', 'MonthlyIncome', 'TotalVisiting'],
    dtype='object'))])

```

```

[60]: # Applying the Transformation
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)

```

```
[61]: pd.DataFrame(X_train).head()
```

```

[61]:
   0  1  2  3  4  5  6  7  8  9  ...  16  17  \
0  1.0  0.0  0.0  1.0  1.0  0.0  0.0  0.0  0.0  0.0  ... -0.7214 -1.020350
1  1.0  0.0  1.0  0.0  1.0  0.0  0.0  0.0  0.0  1.0  ... -0.7214  0.690023
2  1.0  1.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  ... -0.7214 -1.020350
3  1.0  0.0  1.0  0.0  1.0  1.0  0.0  0.0  0.0  1.0  ... -0.7214 -1.020350
4  0.0  0.0  0.0  1.0  0.0  0.0  0.0  0.0  0.0  0.0  ... -0.7214  2.400396

   18  19  20  21  22  23  24  \
0  1.284279 -0.725271 -0.127737 -0.632399  0.679690  0.782966 -0.382245
1  0.282777 -0.725271  1.511598 -0.632399  0.679690  0.782966 -0.459799
2  0.282777  1.771041  0.418708 -0.632399  0.679690  0.782966 -0.245196

```



```

3  1.284279 -0.725271 -0.127737 -0.632399  1.408395 -1.277194  0.213475
4 -1.720227 -0.725271  1.511598 -0.632399 -0.049015 -1.277194 -0.024889

```

```

      25
0 -0.774151
1  0.643615
2 -0.065268
3 -0.065268
4  2.061382

```

[5 rows x 26 columns]

3 Model Training

```

[62]: from sklearn.ensemble import RandomForestClassifier,
      ↪ GradientBoostingClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
      ↪ confusion_matrix, precision_score, recall_score, f1_score, roc_auc_score,
      ↪ roc_curve

```

3.1 An Efficient Way to Train Multiple Models

```

[63]: # Defining all the models
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "Gradient Boost": GradientBoostingClassifier(),
    "Adaboost": AdaBoostClassifier()
}

# Training all the models
for i in range(len(list(models))):
    model = list(models.values())[i]
    model.fit(X_train, y_train)

    # Make Predictions
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    # Training set Performance
    model_train_accuracy = accuracy_score(y_train, y_train_pred)
    model_train_confusion_matrix = confusion_matrix(y_train, y_train_pred)
    model_train_recall = recall_score(y_train, y_train_pred)

```

```

model_train_f1 = f1_score(y_train, y_train_pred)
model_train_precision = precision_score(y_train, y_train_pred)
model_train_rocauc_score = roc_auc_score(y_train, y_train_pred)

# Testing set Performance
model_test_accuracy = accuracy_score(y_test, y_test_pred)
model_test_confusion_matrix = confusion_matrix(y_test, y_test_pred)
model_test_recall= recall_score(y_test, y_test_pred)
model_test_f1 = f1_score(y_test, y_test_pred)
model_test_precision= precision_score(y_test, y_test_pred)
model_test_rocauc_score = roc_auc_score(y_test, y_test_pred)

print(list(models.keys())[i])

# showing the performances

print('Model performance for Training Set')
print("-Accuracy: {:.4f}".format(model_train_accuracy))
print('- F1 score: {:.4f}'.format(model_train_f1))

print('- Precision: {:.4f}'.format(model_train_precision))
print('- Recall: {:.4f}'.format(model_train_recall))
print('- Roc Auc Score: {:.4f}'.format(model_train_rocauc_score))
print(f"-Confusion Matrix:\n {model_test_confusion_matrix}")

print('-----')

print('Model performance for Test set')
print('- Accuracy: {:.4f}'.format(model_test_accuracy))
print('- F1 score: {:.4f}'.format(model_test_f1))
print('- Precision: {:.4f}'.format(model_test_precision))
print('- Recall: {:.4f}'.format(model_test_recall))
print('- Roc Auc Score: {:.4f}'.format(model_test_rocauc_score))
print(f"-Confusion Matrix:\n {model_train_confusion_matrix}")

print('='*35)
print('\n')

```

Logistic Regression

Model performance for Training Set

```

-Accuracy: 0.8460
- F1 score: 0.4234
- Precision: 0.7016
- Recall: 0.3032
- Roc Auc Score: 0.6368

```

```

-Confusion Matrix:
[[762  25]
 [135  56]]
-----
Model performance for Test set
- Accuracy: 0.8364
- F1 score: 0.4118
- Precision: 0.6914
- Recall: 0.2932
- Roc Auc Score: 0.6307
-Confusion Matrix:
[[3087  94]
 [ 508 221]]
=====

```

```

Decision Tree
Model performance for Training Set
-Accuracy: 1.0000
- F1 score: 1.0000
- Precision: 1.0000
- Recall: 1.0000
- Roc Auc Score: 1.0000
-Confusion Matrix:
[[749  38]
 [ 40 151]]
-----
Model performance for Test set
- Accuracy: 0.9202
- F1 score: 0.7947
- Precision: 0.7989
- Recall: 0.7906
- Roc Auc Score: 0.8711
-Confusion Matrix:
[[3181   0]
 [   0 729]]
=====

```

```

Random Forest
Model performance for Training Set
-Accuracy: 1.0000
- F1 score: 1.0000
- Precision: 1.0000
- Recall: 1.0000
- Roc Auc Score: 1.0000
-Confusion Matrix:
[[783   4]

```

[64 127]]

Model performance for Test set

- Accuracy: 0.9305
- F1 score: 0.7888
- Precision: 0.9695
- Recall: 0.6649
- Roc Auc Score: 0.8299
- Confusion Matrix:

[[3181 0]

[0 729]]

=====

Gradient Boost

Model performance for Training Set

- Accuracy: 0.8939
- F1 score: 0.6382
- Precision: 0.8756
- Recall: 0.5021
- Roc Auc Score: 0.7429
- Confusion Matrix:

[[765 22]

[116 75]]

Model performance for Test set

- Accuracy: 0.8589
- F1 score: 0.5208
- Precision: 0.7732
- Recall: 0.3927
- Roc Auc Score: 0.6824
- Confusion Matrix:

[[3129 52]

[363 366]]

=====

Adaboost

Model performance for Training Set

- Accuracy: 0.8478
- F1 score: 0.3847
- Precision: 0.7815
- Recall: 0.2551
- Roc Auc Score: 0.6194
- Confusion Matrix:

[[772 15]

[146 45]]

```

Model performance for Test set
- Accuracy: 0.8354
- F1 score: 0.3586
- Precision: 0.7500
- Recall: 0.2356
- Roc Auc Score: 0.6083
-Confusion Matrix:
[[3129   52]
 [ 543 186]]
=====

```

4 Hyperparameter Tuning

```

[64]: rf_params = {
        "max_depth": [5, 8, 15, None, 10],
        "max_features": [5, 7, "auto", 8],
        "min_samples_split" : [2, 8, 15, 20],
        "n_estimators": [100, 200, 500, 1000]
    }

    gradient_params={
        "loss": ['log_loss', 'deviance', 'exponential'],
        "criterion": ['friedman_mse', 'squared_error', 'mse'],
        "min_samples_split": [1, 8, 15, 20],
        "n_estimators": [100, 200, 500, 1000],
        "max_depth": [5, 8, 15, None, 10]
    }

```

```
[65]: rf_params
```

```

[65]: {'max_depth': [5, 8, 15, None, 10],
      'max_features': [5, 7, 'auto', 8],
      'min_samples_split': [2, 8, 15, 20],
      'n_estimators': [100, 200, 500, 1000]}

```

```
[66]: gradient_params
```

```

[66]: {'loss': ['log_loss', 'deviance', 'exponential'],
      'criterion': ['friedman_mse', 'squared_error', 'mse'],
      'min_samples_split': [1, 8, 15, 20],
      'n_estimators': [100, 200, 500, 1000],
      'max_depth': [5, 8, 15, None, 10]}

```

```
[67]: # Model list for hyperparameter tuning
```

```

randomcv_model = [
    ("RF", RandomForestClassifier(), rf_params),
    ("GD", GradientBoostingClassifier(), gradient_params)
]

```

```

[69]: from sklearn.model_selection import RandomizedSearchCV

model_param = {}
for name, model, params in randomcv_model:
    random = RandomizedSearchCV(estimator=model,
                                param_distributions=params,
                                n_iter=100,
                                cv=3,
                                verbose=2,
                                n_jobs=-1)

    random.fit(X_train, y_train)
    model_param[name] = random.best_params_

for model_name in model_param:
    print(f"----- Best Params for {model_name} -----")
    print(model_param[model_name])

```

```

Fitting 3 folds for each of 100 candidates, totalling 300 fits
Fitting 3 folds for each of 100 candidates, totalling 300 fits
----- Best Params for RF -----
{'n_estimators': 1000, 'min_samples_split': 2, 'max_features': 8, 'max_depth':
None}
----- Best Params for GD -----
{'n_estimators': 500, 'min_samples_split': 20, 'max_depth': 15, 'loss':
'log_loss', 'criterion': 'friedman_mse'}

```

```

[70]: models={

    "Random Forest":RandomForestClassifier(n_estimators=500,min_samples_split=2,
                                             max_features=8,max_depth=15),

    "GradientBoost": GradientBoostingClassifier(n_estimators=500,
↪min_samples_split= 20, max_depth= 15, loss = 'log_loss', criterion=
↪'friedman_mse')
}

# Training all the models
for i in range(len(list(models))):
    model = list(models.values())[i]
    model.fit(X_train, y_train)

    # Make Predictions
    y_train_pred = model.predict(X_train)

```

```

y_test_pred = model.predict(X_test)

# Training set Performance
model_train_accuracy = accuracy_score(y_train, y_train_pred)
model_train_confusion_matrix = confusion_matrix(y_train, y_train_pred)
model_train_recall = recall_score(y_train, y_train_pred)
model_train_f1 = f1_score(y_train, y_train_pred)
model_train_precision = precision_score(y_train, y_train_pred)
model_train_rocauc_score = roc_auc_score(y_train, y_train_pred)

# Testing set Performance
model_test_accuracy = accuracy_score(y_test, y_test_pred)
model_test_confusion_matrix = confusion_matrix(y_test, y_test_pred)
model_test_recall = recall_score(y_test, y_test_pred)
model_test_f1 = f1_score(y_test, y_test_pred)
model_test_precision = precision_score(y_test, y_test_pred)
model_test_rocauc_score = roc_auc_score(y_test, y_test_pred)

print(list(models.keys())[i])

# showing the performances

print('Model performance for Training Set')
print("-Accuracy: {:.4f}".format(model_train_accuracy))
print('- F1 score: {:.4f}'.format(model_train_f1))

print('- Precision: {:.4f}'.format(model_train_precision))
print('- Recall: {:.4f}'.format(model_train_recall))
print('- Roc Auc Score: {:.4f}'.format(model_train_rocauc_score))
print(f"-Confusion Matrix:\n {model_test_confusion_matrix}")

print('-----')

print('Model performance for Test set')
print('- Accuracy: {:.4f}'.format(model_test_accuracy))
print('- F1 score: {:.4f}'.format(model_test_f1))
print('- Precision: {:.4f}'.format(model_test_precision))
print('- Recall: {:.4f}'.format(model_test_recall))
print('- Roc Auc Score: {:.4f}'.format(model_test_rocauc_score))
print(f"-Confusion Matrix:\n {model_train_confusion_matrix}")

print('='*35)
print('\n')

```

Random Forest

Model performance for Training Set

-Accuracy: 0.9995
- F1 score: 0.9986
- Precision: 1.0000
- Recall: 0.9973
- Roc Auc Score: 0.9986
-Confusion Matrix:
[[782 5]
[63 128]]

Model performance for Test set

- Accuracy: 0.9305
- F1 score: 0.7901
- Precision: 0.9624
- Recall: 0.6702
- Roc Auc Score: 0.8319
-Confusion Matrix:
[[3181 0]
[2 727]]

=====

GradientBoost

Model performance for Training Set

-Accuracy: 1.0000
- F1 score: 1.0000
- Precision: 1.0000
- Recall: 1.0000
- Roc Auc Score: 1.0000
-Confusion Matrix:
[[781 6]
[36 155]]

Model performance for Test set

- Accuracy: 0.9571
- F1 score: 0.8807
- Precision: 0.9627
- Recall: 0.8115
- Roc Auc Score: 0.9019
-Confusion Matrix:
[[3181 0]
[0 729]]

=====


```
[71]: ## Plot ROC AUC Curve
from sklearn.metrics import roc_auc_score, roc_curve
plt.figure()

# Add the models to the list that you want to view on the ROC plot
auc_models = [
{
    'label': "GradientBoost",
    'model': GradientBoostingClassifier(n_estimators=500, min_samples_split=
↪20, max_depth= 15, loss = 'log_loss', criterion= 'friedman_mse'),
    'auc': 0.9019
},
]

# create loop through all model
for algo in auc_models:
    model = algo['model']
    model.fit(X_train, y_train)

    fpr, tpr, thresholds = roc_curve(y_test, model.predict_proba(X_test)[: ,1])

    plt.plot(fpr, tpr, label='%s ROC (area = %0.2f)' % (algo['label'],
↪algo['auc']))

# Custom settings for the plot
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('1-Specificity(False Positive Rate)')
plt.ylabel('Sensitivity(True Positive Rate)')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```

